
Linked Open Data Application for Movie Knowledge Graph

Nguyen Lam Thai
HUST

Thai.NL261068M@sis.hust.edu.vn

Dinh Trong Nghia
HUST

Nghia.DT261074M@sis.hust.edu.vn

Nguyen Trong Tuan
HUST

Tuan.NT261067M@sis.hust.edu.vn

Vuong Dinh Minh
HUST

Minh.VD261073M@sis.hust.edu.vn

Abstract

tbd

1 Introduction

The rapid growth of digital media has produced a vast amount of structured information about movies, actors, directors, production companies, genres, and ratings. Platforms such as The Movie Database (TMDB) and IMDb maintain millions of records, and this data is widely used for search, recommendation, and analytics. However, it is often scattered across isolated databases and tabular datasets, each with its own schema, identifiers, and access method. As a result, it is difficult to query across sources, to connect semantically related entities or to ask questions that span several entity types. Knowledge graphs address this problem by representing entities and their relationships in an interconnected graph structure, supporting integration, exploration, and reasoning across heterogeneous information sources [Hogan et al., 2021]. This perspective is consistent with the Linked Data principles of using dereferenceable identifiers and links to related resources [Berners-Lee, 2006].

This work develops a Linked Open Data (LOD) application centered on the movie domain. It covers movies and related entities, including people, organizations, genres, countries, languages, and ratings, together with relationships such as cast membership, directorship, production affiliation, and publication date. The resulting knowledge graph is designed as a reusable and interoperable semantic resource, while remaining focused on a representative subset of the domain that can be modeled consistently using RDF, OWL, and Schema.org vocabulary [Schema.org, 2025, W3C RDF Working Group, 2014]. Reusing Schema.org terms rather than inventing new ones lowers the barrier for other applications to interpret the data.

We demonstrate a complete pipeline for building a movie knowledge graph in accordance with LOD principles. As illustrated in Figure 1, the proposed workflow consists of six stages: (1) defining an ontology for the movie domain; (2) collecting relevant movie data from TMDB; (3) transforming the collected CSV data into RDF in Turtle notation using stable URIs; (4) building an OWL ontology that defines the domain classes, properties, and restrictions; (5) establishing links to DBpedia and Wikidata to improve interoperability and achieve a higher level of Linked Open Data integration; and (6) providing a SPARQL interface through Apache Jena Fuseki for querying the resulting graph [W3C SPARQL Working Group, 2013]. As an optional stage, we provide a static web application, hosted on GitHub Pages in the style of DBpedia, to support browsing and exploration of the generated data.

The remainder of this report is organized as follows. Section 2 details our method, defines the ontology for the movie domain, and specifies the evaluation protocol. Section 3 presents the data

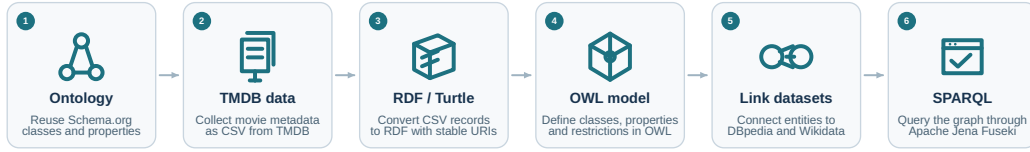


Figure 1: Overview of our method.

collection and transformation. Section 4 introduces the ontology construction and entity linking. Section 5 describes the evaluation. Section 6 discusses limitations and potential future work. Finally, section 7 concludes.

2 Methodology

Figure 1 summarizes the proposed pipeline for constructing and exposing a movie knowledge graph. Movie data are collected from TMDB and stored in CSV files. These tabular records are transformed into Turtle RDF using stable resource URIs. The OWL ontology then adds class, property, and restriction semantics and supports OWL-RL inference. Movie entities are linked to DBpedia and Wikidata using identifiers, titles, release years, and related matching criteria. Apache Jena Fuseki provides a browser-accessible SPARQL endpoint for querying the graph. The optional static web application publishes generated resource pages and ontology views for human browsing.

Here, we describe the first step of the workflow (i.e., defining the ontology for the movie domain) and specify the evaluation protocol, which assesses the resulting knowledge graph by executing representative SPARQL queries. The remaining stages are described in their corresponding sections.

Movie ontology We construct the movie ontology by reusing Schema.org classes and properties to represent movies and their related entities. Table 1 lists all Schema.org terms used in the ontology [Schema.org, 2025]. The classes model movies, people, roles, descriptive entities, organizations, and ratings, while the properties describe movie metadata and relationships, including titles, identifiers, release dates, duration, genres, languages, countries, production companies, ratings, directors, actors, and acting roles. This vocabulary provides a shared semantic model while allowing the data to be serialized as RDF and constrained or extended with OWL.

Evaluation protocol The evaluation protocol queries the graph using SPARQL. The query set covers basic retrieval, such as listing movies, people, and directors, as well as multi-condition questions involving ratings, genres, release years, and cast members. Additional queries test the value of OWL-RL reasoning by asking for inferred relationships, such as a person’s directed movies or a production company’s country. Finally, federated queries retrieve information about a movie from DBpedia or Wikidata through its established links. Queries are executed against the local graph, with reasoning enabled when required, and against the corresponding remote endpoint for external-source questions.

3 Data collection and transformation

3.1 Data collection

The movie dataset is collected from TMDB, a public source of movie metadata. The collection focuses on the first five pages of TMDB’s popular-movie listing, which return 100 movies in total. For each listed movie, detailed metadata and credit information are retrieved. This produces a bounded snapshot of the popular-movie catalogue rather than a complete crawl of TMDB.

The collected movie attributes include the TMDB and IMDb identifiers, title, synopsis, original language, release date, runtime, average vote, and vote count. The collection also records related entities: genres, production companies, production countries, spoken languages, cast members, and

Table 1: Schema.org classes and properties reused in the movie ontology.

Category	Schema.org terms and use
Classes	Movie (movie); CreativeWork (movie superclass); Person (actor, director, or crew member); Role (reified acting role); Genre (movie genre); Country (movie or company country); Language (movie language); Organization (production company); AggregateRating (movie rating); Rating, Thing, and Intangible (reused superclasses).
Object properties	actor (movie or role to actor); aggregateRating (movie to rating); contributor (movie to non-director crew member); countryOfOrigin (movie to country); director (movie to director); genre (movie to genre); inLanguage (movie to language); location (company to country); productionCompany (movie to company); sameAs (movie to an equivalent external resource).
Datatype properties	abstract (movie synopsis); characterName (character portrayed in a role); datePublished (release date); duration (runtime); identifier (TMDb or IMDb identifier); name (entity name); position (cast order); ratingCount (number of votes); ratingValue (rating value); roleName (crew job title).

Table 2: Structure of the collected movie data.

Data group	Information represented
Movie metadata	Identifiers, title, synopsis, language, release date, runtime, and rating information.
Descriptive entities	Genres, production companies, production countries, and spoken languages.
Credits	Cast members, character names, cast order, directors, and person identifiers.
Relationships	Movie-to-genre, movie-to-company, movie-to-country, and movie-to-language associations.

directors. Cast records retain the first ten credited actors, their character names, and their order in the cast. Crew records retain directors and their person identifiers. This selection prioritizes the information needed for movie retrieval, recommendation-style queries, and relationship queries over the entire credit list.

The result is organized as entity tables and relationship tables, under the CSV format. Entity tables store one row per movie, genre, company, country, language, or person reference, while relationship tables preserve many-to-many associations such as movie–genre and movie–production-company links. This separation avoids repeating descriptive values and provides a convenient intermediate representation for RDF generation. Text values are trimmed, missing values are represented as empty fields, and all files are written using UTF-8 encoding.

Table 2 summarizes this organization. The first row contains the central movie records, the second covers reusable descriptive entities, and the third captures the available credit information. The final row distinguishes relationship records from entity records; these links are later converted into RDF predicates between stable resources.

This organization also makes the limitations of the snapshot explicit. Movies can occur more than once in the popular-movie results, and the available metadata depends on the completeness of the source records. In addition, restricting the cast to ten entries reduces the size of the graph but does not represent the full cast of every movie.

3.2 Data transformation

The tabular data are transformed into an RDF graph using Schema.org vocabulary. Each identified entity receives a stable URI composed of a project base URI, an entity type, and its source identifier. For example, movie, person, genre, company, country, and language resources are separated by their respective URI paths. This URI design preserves the identity of entities across files and enables relationships to be represented as links between resources rather than duplicated text values.

Table 3: Qualitative RDF snapshot for “Spider-Man: Brand New Day”.

```

<https://example.org/movie/969681> a schema:Movie ;
  schema:name "Spider-Man: Brand New Day" ;
  schema:datePublished "2026-07-29"^^xsd:date ;
  schema:duration "PT2H25M"^^xsd:duration ;
  schema:aggregateRating [ a schema:AggregateRating ;
    schema:ratingCount 2795 ;
    schema:ratingValue 7.86 ] ;
  schema:genre <https://example.org/genre/12>,
    <https://example.org/genre/28>,
    <https://example.org/genre/878> ;
  schema:director <https://example.org/person/1144604> ;
  schema:sameAs <https://www.themoviedb.org/movie/969681>,
    <https://www.imdb.com/title/tt22084616/> .

```

```

<https://example.org/movie/969681>
  schema:actor [ a schema:Role ;
    schema:actor <https://example.org/person/1136406> ;
    schema:characterName "Peter Parker / Spider-Man" ;
    schema:position 0 ] .

```

Movies are represented as instances of `schema:Movie`. Their descriptive fields are mapped to properties such as `schema:name`, `schema:abstract`, `schema:datePublished`, and `schema:duration`. Related entities are represented as typed resources: genres as `schema:Genre`, people as `schema:Person`, production companies as `schema:Organization`, countries as `schema:Country`, and languages as `schema:Language`. Relationship tables consequently become graph edges such as `schema:genre`, `schema:productionCompany`, `schema:countryOfOrigin`, and `schema:inLanguage`.

Values are assigned RDF datatypes where appropriate. Release dates use `xsd:date`, runtimes are represented as `xsd:duration`, rating values use `xsd:decimal`, and vote counts and cast positions use `xsd:integer`. Empty source values are omitted instead of being emitted as empty literals. Ratings are represented by an anonymous `schema:AggregateRating` resource so that the rating value and vote count remain grouped. Similarly, actor information is reified as `schema:Role` resources when character names or cast positions are available.

Table 3 is a qualitative example rather than a complete listing of the resource. The first group of statements describes the movie itself and demonstrates how dates, durations, and ratings are typed. The genre and director objects are URI links to other resources in the graph, whereas the `schema:sameAs` objects link the local resource to its TMDb and IMDb counterparts. The final group shows why a role resource is useful: it preserves both the actor identity and the character name while also retaining the cast position.

4 Ontology construction and entity linking

4.1 Ontology construction

The ontology provides the semantic layer of the movie knowledge graph. It is written by reusing Schema.org terms and adding OWL axioms that describe the intended meaning of movie entities and their relationships. The design covers movies, people, genres, production organizations, countries, languages, ratings, and reified acting roles. In addition to naming classes and properties, the ontology specifies domains, ranges, inverse relationships, property chains, and cardinality constraints. These axioms support validation of the graph and inference over relationships that are not explicitly stored.

Table 4 presents four representative excerpts from the ontology. The first row defines `schema:Movie` as a class specialized from `schema:CreativeWork`. The second row shows a datatype property, `schema:datePublished`, whose values must be dates. The third row shows the object property `schema:director`, which connects a movie resource to a person resource. The final row illustrates a restriction requiring every movie to have at least one name. Together, these examples show that the ontology describes both the vocabulary of the graph and constraints on its data.

Table 4: Representative excerpts from the movie ontology.

Element	OWL excerpt	Role in the graph
Class	<code>schema:Movie a owl:Class; rdfs:subClassOf schema:CreativeWork .</code>	Identifies movie resources and places them under the Schema.org creative-work hierarchy.
Datatype property	<code>schema:datePublished a owl:DatatypeProperty; rdfs:domain schema:Movie; rdfs:range xsd:date .</code>	Represents a movie release date and constrains its value to <code>xsd:date</code> .
Object property	<code>schema:director a owl:ObjectProperty; rdfs:domain schema:Movie; rdfs:range schema:Person .</code>	Links a movie to a person who directed it.
Restriction	<code>schema:Movie rdfs:subClassOf [a owl:Restriction; owl:onProperty schema:name; owl:minCardinality 1] .</code>	Requires every movie to have at least one name.

Table 5: Examples of candidate scoring and linking decisions.

Movie	Matched by	Evidence	Score	Status
Spider-Man: Brand New Day	Identifier search	TMDB + IMDb + release year	1.00	Approved
The Furious	Identifier search	TMDB + IMDb; release-year mismatch	0.00	Rejected
The Fix	Identifier search	TMDB + IMDb; release year missing	0.50	Review

The ontology also defines the inverse property `kg:directedMovie` for retrieving movies from a director and a property chain that derives a movie’s production country from its production company. Disjointness axioms distinguish entity categories such as movies, people, and organizations, while restrictions on ratings and acting roles limit ambiguous or incomplete representations. These modelling decisions make the graph suitable for semantic querying rather than simple metadata lookup.

4.2 Entity linking

Entity linking connects local movie resources to equivalent resources in Wikidata and DBpedia. Candidate entities are first searched using TMDB and IMDb identifiers. When identifier-based search does not produce a candidate, the movie title and release year are used. Each candidate is then compared with the local record. An identifier match combined with a matching release year receives the highest score, while a release-year mismatch is rejected. Candidates with incomplete or weaker evidence are retained for manual review.

Table 5 gives three representative outcomes from the candidate-matching process. The accepted example, Spider-Man: Brand New Day, matches both identifiers and the release year, producing a score of 1.00. The rejected candidate for The Furious has matching identifiers but a conflicting release year, so it receives a score of 0.00. The The Fix candidate has matching identifiers but no comparison release year and is therefore assigned a score of 0.50 and marked for review. The table illustrates why the score is used together with explicit matching evidence and a status rather than being treated as an unqualified identity claim.

Only approved candidates are materialized as external links. The resulting links use `owl:sameAs`, asserting that the local URI and the external resource identify the same movie. Table 6 shows a qualitative excerpt for Spider-Man: Brand New Day. One local resource is linked to both a Wikidata entity and a DBpedia resource, allowing subsequent queries to follow either link to complementary

Table 6: Excerpt of the generated external entity links.

```
<https://example.org/movie/969681>
owl:sameAs
<http://dbpedia.org/resource/Spider-Man:_Brand_New_Day>,
<http://www.wikidata.org/entity/Q113244935> .
```

Table 7: SPARQL query for basic movie retrieval.

```
SELECT DISTINCT ?movie_id ?movie_name
WHERE {
  ?movie_id a schema:Movie ;
  schema:name ?movie_name .
}
LIMIT 20
```

linked-data sources. The excerpt also demonstrates that the local URI remains the stable identifier within the project, while the external URIs preserve the identity of the corresponding resources.

This linking stage extends the locally generated RDF graph with cross-dataset identity links. It therefore supports federated exploration and five-star Linked Open Data principles while keeping uncertain matches separate from automatically approved assertions.

5 Evaluation

The knowledge graph is evaluated through representative SPARQL queries designed to demonstrate increasing expressiveness. The query collection is organized into four categories: basic retrieval, structured filtering and aggregation, reasoning-based queries, and cross-knowledge-graph queries. Results below are trimmed to a compact representative sample where the original query returns more records.

5.1 Basic retrieval

The first category tests direct access to resources and their descriptive properties. The question asked in this example is: *Which movies are present in the knowledge graph?* The SPARQL pattern in Table 7 retrieves movie names and local identifiers without requiring joins beyond the movie type and name properties. Table 8 shows the first five returned records. These results demonstrate that the graph can provide a simple catalogue view while preserving stable resource identifiers for subsequent queries.

5.2 Structured filtering and aggregation

The second category combines several graph patterns with filtering and ordering. The question asked here is: *Which are the highest-rated Action movies released since 2020 that starred Tom Holland?* The SPARQL pattern in Table 9 selects movies that satisfy these conditions and orders them by rating value. Table 10 reports all three matching results in the generated dataset. Unlike the basic retrieval query, this query traverses movie–genre, movie–role–actor, and movie–rating relationships simultaneously, while also applying date and value conditions.

Table 10 shows that the same graph can support selection, joins, aggregation-ready rating values, and ranking in one query. The result is therefore more expressive than a flat lookup because the answer depends on several connected entity types and constraints.

5.3 Reasoning-based queries

The third category evaluates facts that are derived from the OWL ontology rather than stored as direct movie attributes. The question is: *Which country is the production company of Avengers: Endgame from?* The SPARQL pattern in Table 11 follows the inferred property connecting the movie to its production country. OWL-RL reasoning is enabled before query execution. The result in

Table 8: Representative results for basic movie retrieval.

Movie	Local resource
Hope	movie/1058424
Supergirl	movie/1081003
Backrooms	movie/1083381
Zootopia 2	movie/1084242
Toy Story 5	movie/1084244

Table 9: SPARQL query for multi-condition filtering and ranking.

<pre> SELECT ?movie_name ?genre ?date ?actor_name ?rating_value WHERE { ?id a schema:Movie ; schema:name ?movie_name ; schema:datePublished ?date ; schema:genre ?g ; schema:actor ?a ; schema:aggregateRating ?rating . ?g schema:name "Action" . ?a schema:actor ?actor_id . ?actor_id schema:name "Tom Holland" . ?rating schema:ratingValue ?rating_value . FILTER (?date >= "2020-01-01"^^xsd:date) } ORDER BY DESC(?rating_value) </pre>

Table 12 identifies the United States of America, although the query does not directly use the movie’s country-of-origin property.

Table 12 illustrates the role of the ontology in query answering. Without the property-chain inference, the requested movie–country relationship would not be available through the same query pattern. This category consequently demonstrates semantic enrichment rather than only the retrieval of explicitly asserted triples.

5.4 Cross-knowledge-graph queries

The final category uses the `owl:sameAs` links to retrieve additional information from an external knowledge graph. The question is: *What additional information about The Odyssey can be retrieved from DBpedia?* The SPARQL pattern in Table 13 follows the local `owl:sameAs` link and retrieves properties of the external resource. Since the complete external description contains many properties, Table 14 shows selected results that complement the local graph, including the film type, director, production company, budget, and a starring actor.

Table 14 demonstrates that the local graph can act as an entry point to information not collected in the local movie snapshot. The local resource supplies the identity link, while DBpedia contributes additional descriptive properties such as budget and production credits. Taken together, the four query categories show a progression from direct lookup to graph traversal, inference, and linked-data integration. They demonstrate the practical expressiveness of the constructed knowledge graph without interpreting the difficulty labels as formal research metrics.

6 Discussion

Limitations

Future work

Table 10: Results for multi-condition filtering and ranking.

Movie	Year	Rating	Actor
The Odyssey	2026	8.000	Tom Holland
Spider-Man: No Way Home	2021	7.947	Tom Holland
Spider-Man: Brand New Day	2026	7.860	Tom Holland

Table 11: SPARQL query using an inferred production-country relation.

<pre> SELECT ?country ?name WHERE { <https://example.org/movie/299534> kg:productionCountry ?country . ?country schema:name ?name . } ORDER BY ?name </pre>

7 Conclusion

References

- Tim Berners-Lee. Linked data. <https://www.w3.org/DesignIssues/LinkedData.html>, 2006. Accessed 2026-10-05.
- Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d’Amato, Gerard de Melo, Claudio Gutierrez, Sabrina Kirrane, Jose Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, Axel-Cyrille Ngonga Ngomo, Axel Polleres, Sabbir M. Rashid, Anisa Rula, Lukas Schmelzeisen, Juan Sequeda, Steffen Staab, and Antoine Zimmermann. Knowledge graphs. *ACM Computing Surveys*, 54(4), 2021. doi: 10.1145/3447772.
- Schema.org. Getting started with schema.org. <https://schema.org/docs/gs.html>, 2025. Accessed 2026-10-05.
- W3C RDF Working Group. Rdf 1.1 concepts and abstract syntax. <https://www.w3.org/TR/rdf11-concepts/>, 2014. W3C Recommendation; accessed 2026-10-05.
- W3C SPARQL Working Group. Sparql 1.1 query language. <https://www.w3.org/TR/sparql11-query/>, 2013. W3C Recommendation; accessed 2026-10-05.

Table 12: Result of an OWL-RL reasoning-based query.

Movie	Inferred production country
Avengers: Endgame	United States of America

Table 13: SPARQL query for retrieving linked external information.

```

SELECT ?p ?o
WHERE {
  <https://example.org/movie/1368337>
    owl:sameAs ?remote_entity .
  ?remote_entity ?p ?o .
}

```

Table 14: Selected information retrieved from DBpedia for “The Odyssey”.

Property	Retrieved value
Type	dbpedia-ontology:Film
Director	Christopher Nolan
Production company	Universal Pictures
Budget	250,000,000
Starring actor	Tom Holland